Credit Risk Modelling and Lending Decision Analysis

Predictive modelling, expected-loss analysis, and SHAP interpretability for home-equity loan default risk

This report presents the final project results in a code-hidden, shareable format. It loads saved project outputs and focuses on the analytical narrative, model comparison, economic decision trade-offs, interpretation, conclusions, and recommendations.

Project Overview

The project uses the HMEQ home-equity loan dataset to predict whether a borrower is likely to default or become severely delinquent. The target variable is `BAD`:

- `BAD = 1`: borrower defaulted or was severely delinquent
- `BAD = 0`: borrower repaid the loan

The goal is not only to compare predictive models, but also to translate predictions into lending decisions. This means evaluating the cost of approving risky borrowers and the opportunity cost of rejecting good borrowers.

Business Problem

A lender faces two types of decision error:

- **False negative:** the model approves a borrower who later defaults. This exposes the lender to credit losses.
- **False positive:** the model rejects a borrower who would have repaid. This creates missed interest income and may harm customer acquisition.

Because these errors have different business costs, accuracy alone is not a sufficient evaluation metric. The project therefore combines standard classification metrics with an expected-loss framework.

Data and Target Variable

The dataset contains loan amount, mortgage balance, property value, employment information, credit history, and debt-to-income variables. Missingness is meaningful in this dataset, so the final modelling scripts treat missing categorical values as an explicit `"Missing"` category and add missingness indicators for selected numerical fields.

	Variable	Description
0	BAD	Default indicator: 1 = default/severe delinque...
1	LOAN	Amount of loan approved
2	MORTDUE	Amount due on existing mortgage
3	VALUE	Current property value
4	REASON	Reason for loan request
5	JOB	Applicant job type
6	YOJ	Years at present job
7	DEROG	Number of major derogatory reports
8	DELINQ	Number of delinquent credit lines
9	CLAGE	Age of oldest credit line in months
10	NINQ	Number of recent credit inquiries
11	CLNO	Number of existing credit lines
12	DEBTINC	Debt-to-income ratio

Exploratory Analysis Snapshot

The EDA showed class imbalance, substantial missingness in selected variables, skewed financial variables, and visible relationships between default status and credit-history/debt-burden variables. The plots below are selected from the descriptive-analysis outputs.

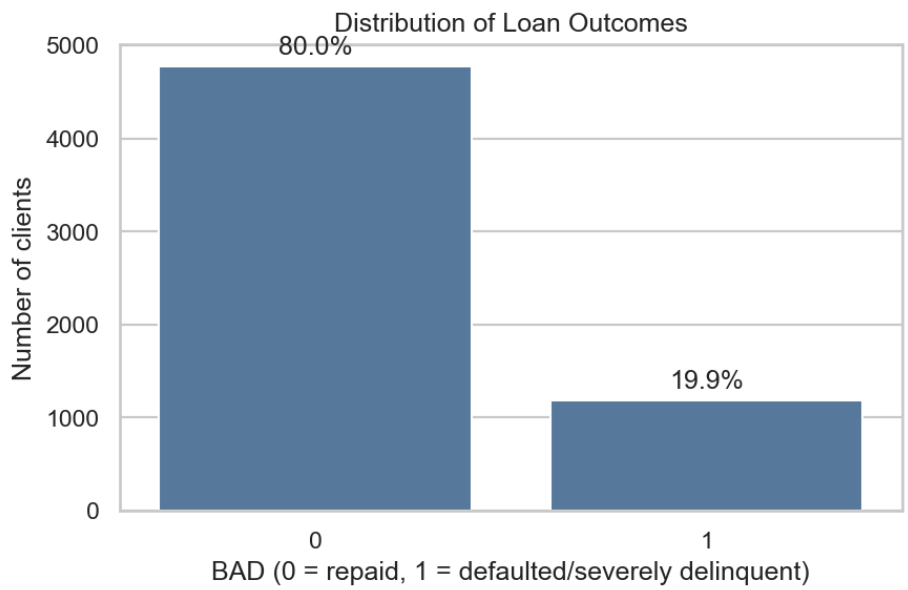
Target balance

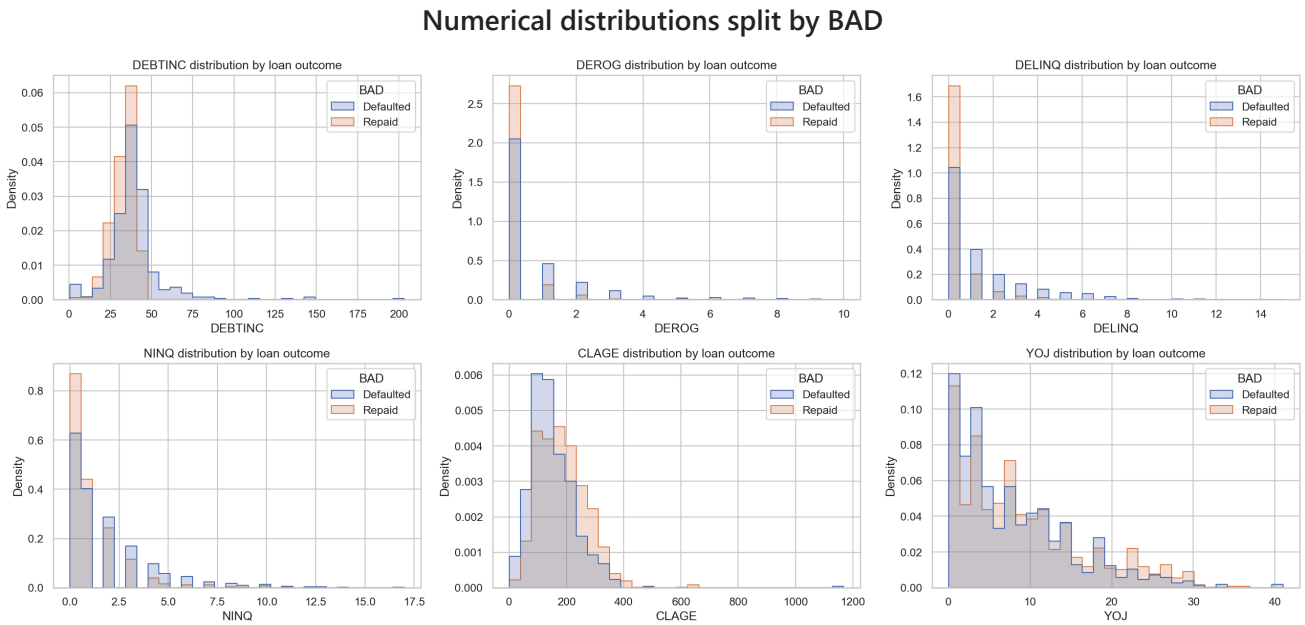
	BAD	class_label	count	percent
0	0	Repaid	4771	80.0500
1	1	Defaulted / severely delinquent	1189	19.9500

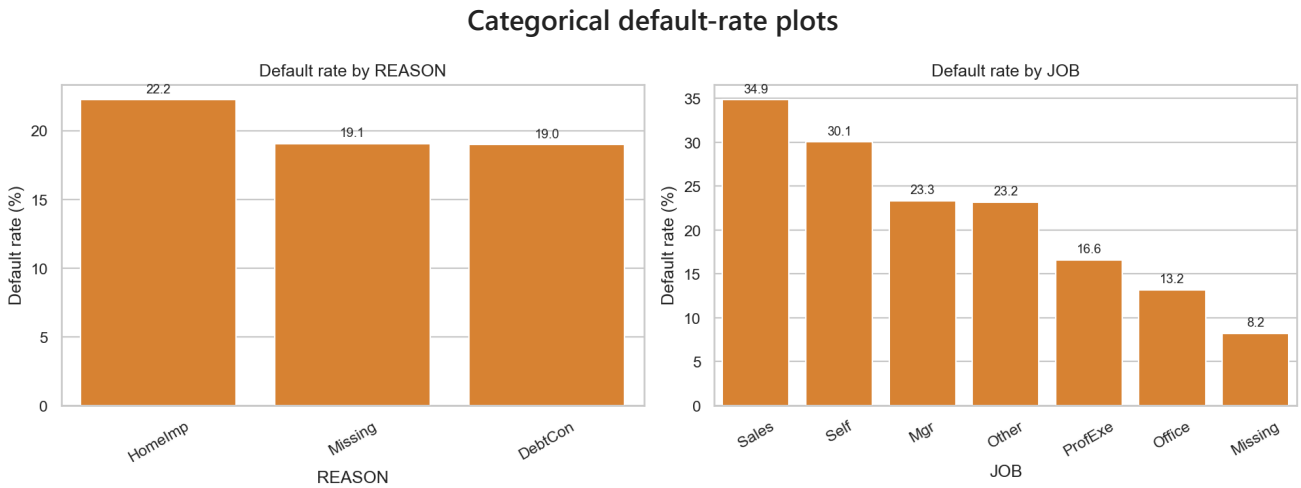
Missing-value summary

	missing_count	missing_%
0	1,267.0000	21.2600
1	708.0000	11.8800
2	580.0000	9.7300
3	518.0000	8.6900
4	515.0000	8.6400
5	510.0000	8.5600
6	308.0000	5.1700
7	279.0000	4.6800
8	252.0000	4.2300
9	222.0000	3.7200
10	112.0000	1.8800
11	0.0000	0.0000
12	0.0000	0.0000

Target distribution







Modelling Strategy

The modelling stage compares interpretable baselines and stronger non-linear models:

- Logistic regression
- Decision tree
- Random forest
- Gradient Boosting
- AdaBoost
- XGBoost

The revised scripts use a consistent, leakage-free approach:

- split data before fitting preprocessing steps;
- stratified train/test split with `random_state=42`;

- median imputation for numerical predictors;
- explicit `"Missing"` category for categorical predictors;
- one-hot encoding for `REASON` and `JOB`;
- model-specific tuning using cross-validation.

Main Predictive Performance

The main predictive comparison uses the saved test-set metrics from the model scripts. The project has generally prioritised ROC-AUC because it measures ranking quality across thresholds, while F1, precision, and recall help assess the positive class `BAD = 1`.

Main model comparison from the XGBoost script

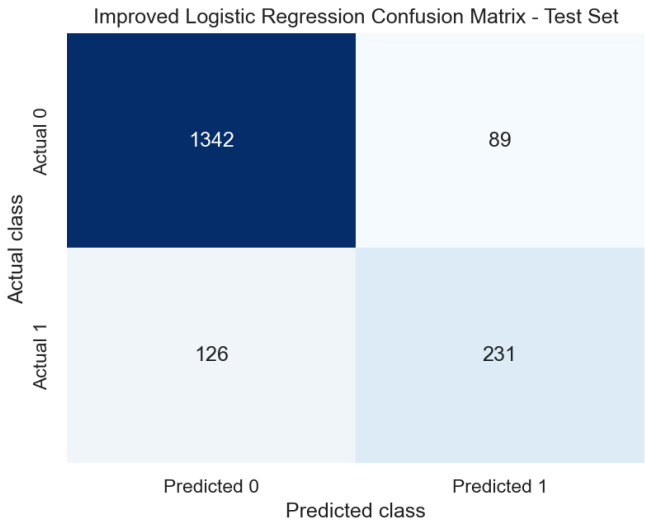
	model	accuracy	precision_BAD_1	recall_BAD_1	f1_BAD_1	roc_auc	average_precision	fa
5	XGBoost	0.9239	0.7962	0.8319	0.8137	0.9641	0.8971	
3	Gradient Boosting	0.9178	0.7720	0.8347	0.8022	0.9633	0.8975	
2	Random Forest	0.9049	0.7416	0.8039	0.7715	0.9554	0.8348	
4	AdaBoost	0.9083	0.7749	0.7619	0.7684	0.9506	0.8641	
0	Logistic Regression	0.8798	0.7219	0.6471	0.6824	0.8903	NaN	
1	Decision Tree	0.8697	0.6845	0.6443	0.6638	0.8835	NaN	

Interpretation. XGBoost is the strongest model on the main predictive metrics, with the highest ROC-AUC and F1 among the saved model comparisons. Gradient Boosting is very close and has slightly higher recall in the selected-threshold comparison, which means the ranking is not completely one-dimensional. This is why the later decision analysis is useful: it connects model errors to business costs.

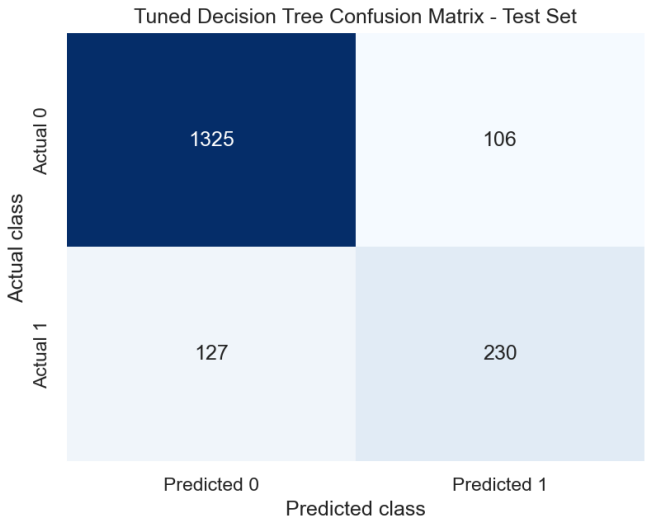
Confusion Matrices and Classification Trade-Offs

Confusion matrices show the practical consequences of a threshold. For default prediction, false negatives are especially important because they represent likely defaulters who would be approved by the model.

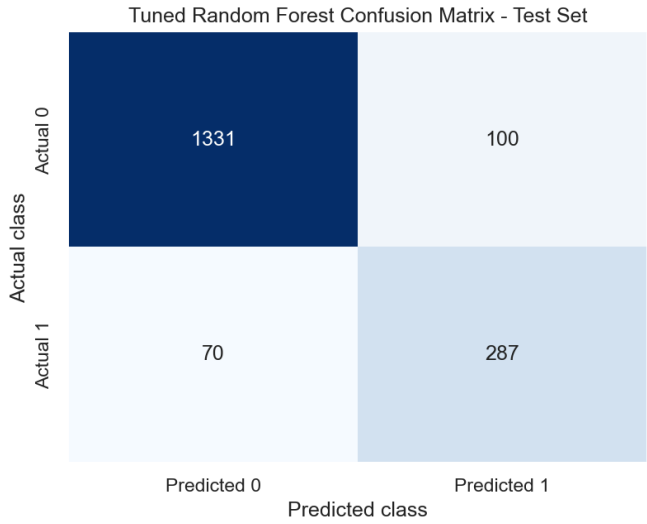
Logistic regression confusion matrix



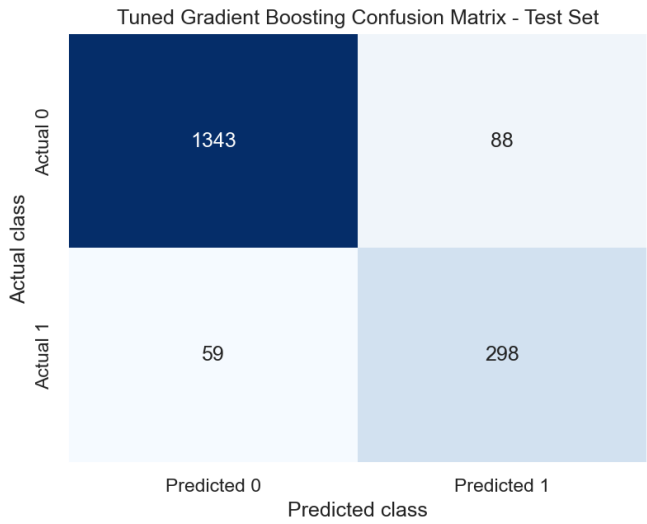
Decision tree confusion matrix



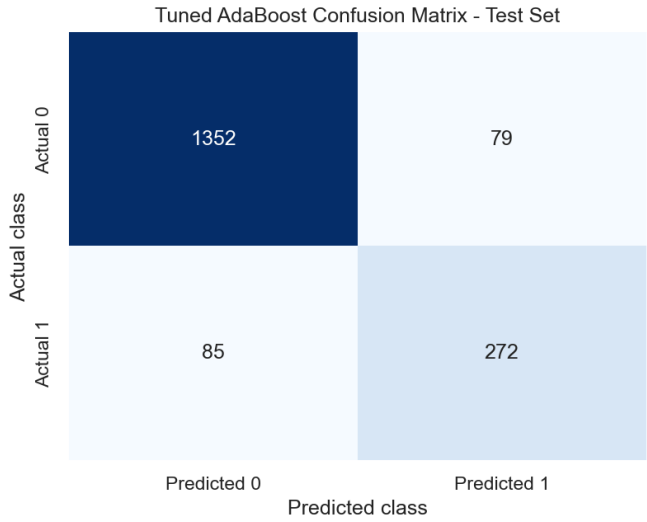
Random forest confusion matrix



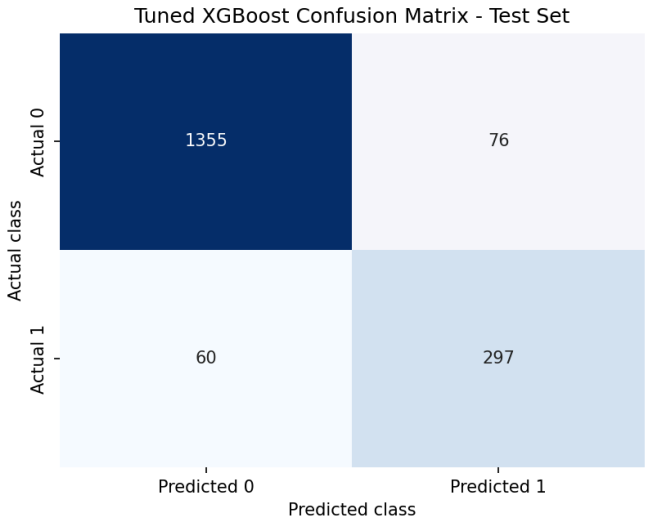
Gradient Boosting confusion matrix



AdaBoost confusion matrix



XGBoost confusion matrix

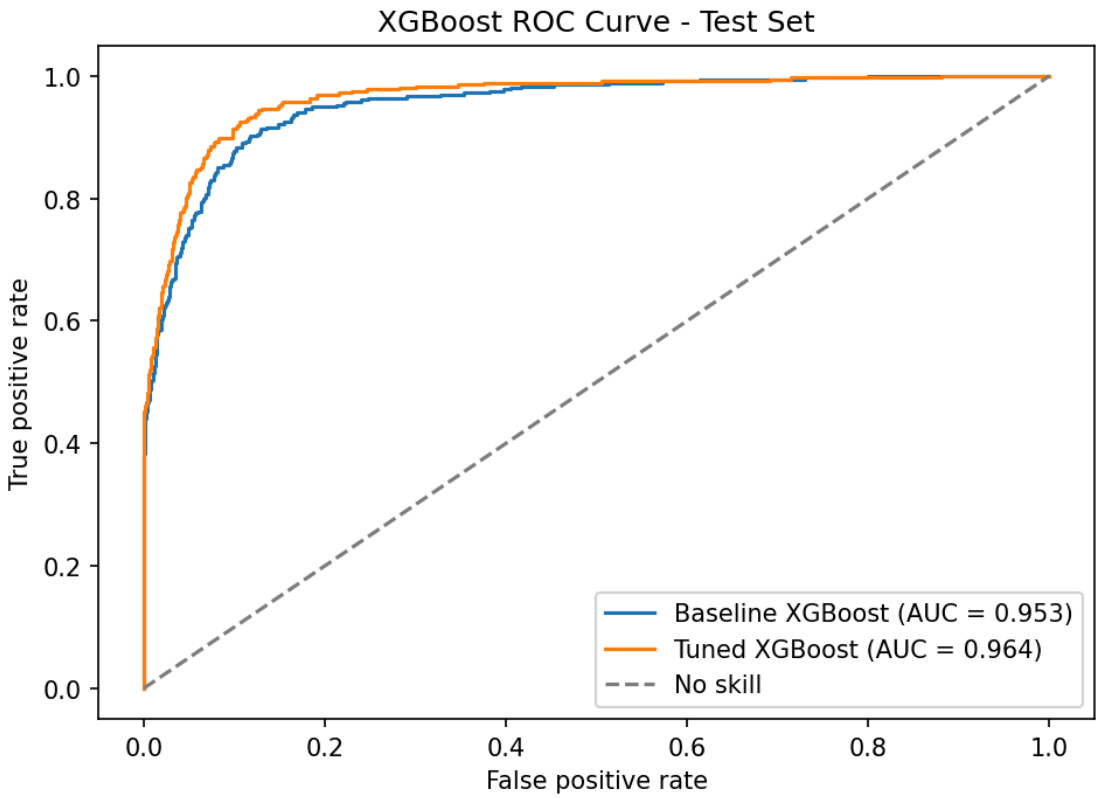


Interpretation. The stronger ensemble models reduce missed defaulters relative to simpler baselines, but threshold choice changes the balance between recall and precision. A lower threshold catches more defaulters, but also rejects more borrowers who would have repaid.

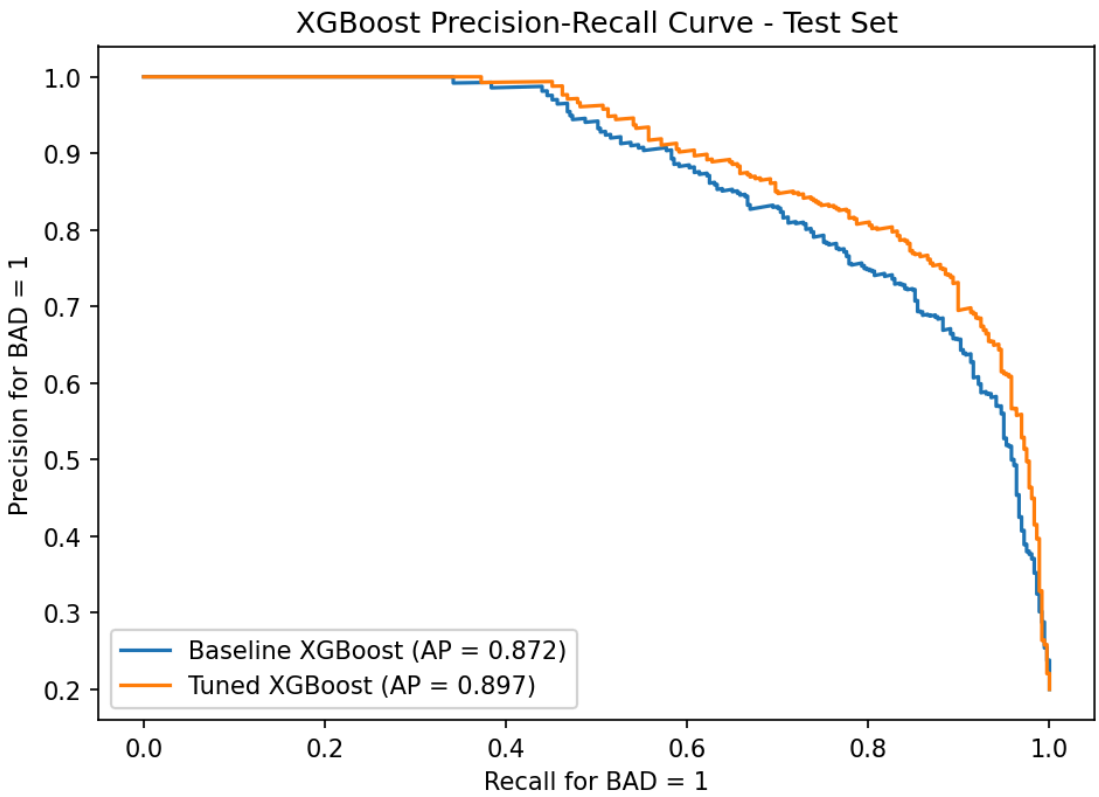
ROC-AUC and Threshold Performance

ROC and precision-recall curves show how model performance changes across thresholds. The ensemble models have stronger separation between defaulters and non-defaulters than the simpler baselines.

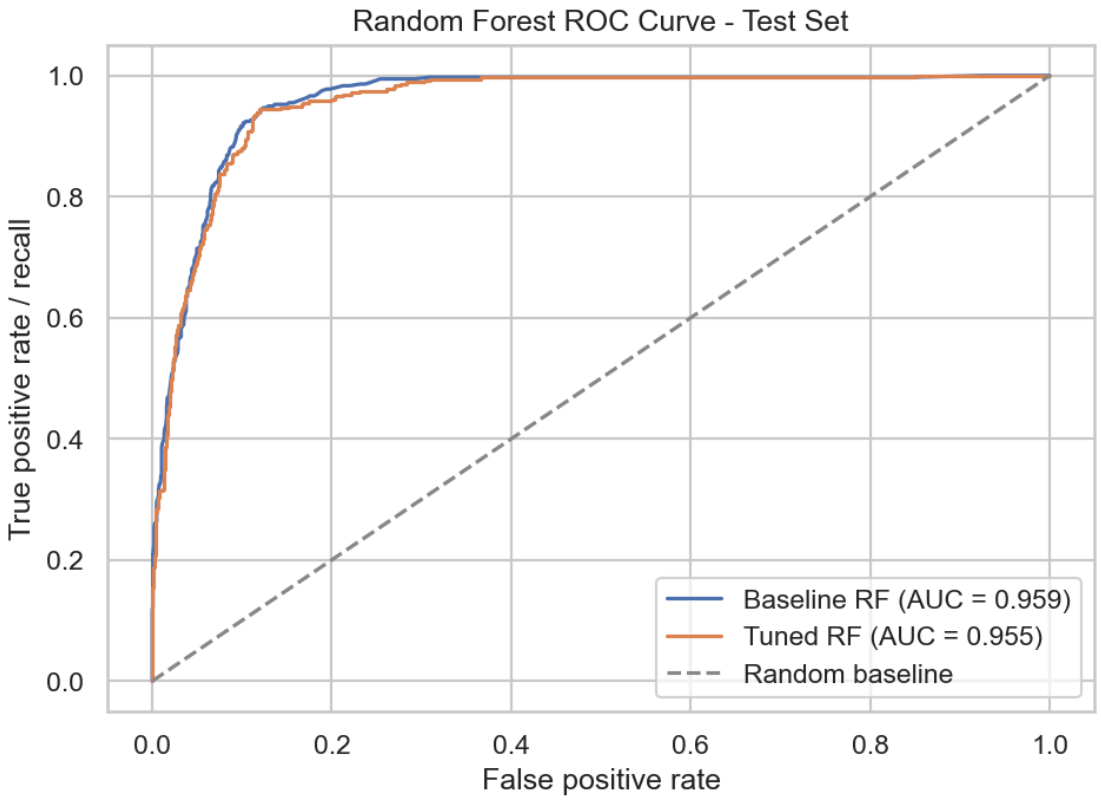
XGBoost ROC curve



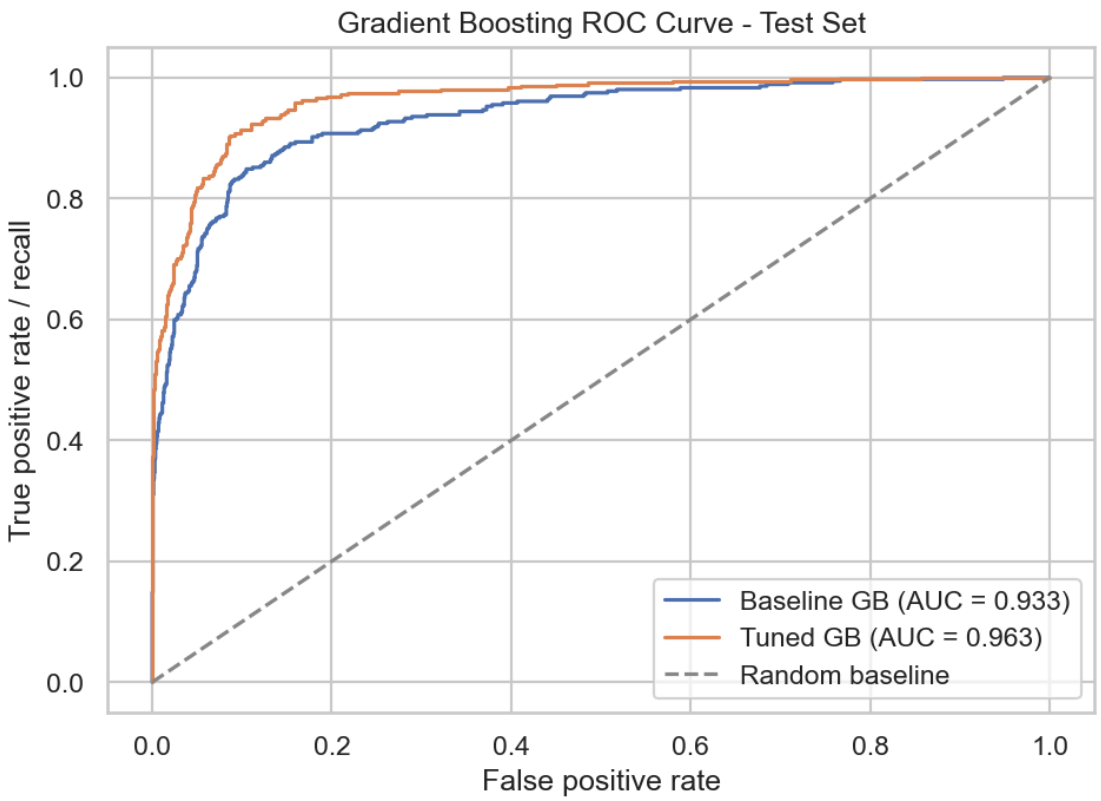
XGBoost precision-recall curve



Random forest ROC curve



Gradient Boosting ROC curve



XGBoost threshold analysis

	accuracy	precision_BAD_1	recall_BAD_1	f1_BAD_1	roc_auc	average_precision	false_positive
72	0.9257	0.8042	0.8293	0.8166	0.9599	0.8924	16
70	0.9252	0.8009	0.8317	0.8160	0.9599	0.8924	17
74	0.9257	0.8071	0.8245	0.8157	0.9599	0.8924	16
77	0.9262	0.8126	0.8185	0.8156	0.9599	0.8924	15
75	0.9257	0.8078	0.8233	0.8155	0.9599	0.8924	16
71	0.9250	0.8007	0.8305	0.8153	0.9599	0.8924	17
69	0.9247	0.7991	0.8317	0.8151	0.9599	0.8924	17
68	0.9245	0.7968	0.8341	0.8150	0.9599	0.8924	17
78	0.9259	0.8124	0.8173	0.8149	0.9599	0.8924	15
65	0.9238	0.7907	0.8401	0.8147	0.9599	0.8924	18

Interpretation. The model probability threshold should be treated as a business decision parameter, not a fixed statistical default. The best F1 threshold is not necessarily the best expected-loss threshold, because the economic cost of a false negative is much larger than the cost of a false positive in the baseline lending scenario.

Expected-Loss Analysis

The expected-loss analysis translates model errors into economic costs:

- False negative cost: borrower defaults after being approved. Cost equals unrecovered loan value.
- False positive cost: borrower would have repaid but is rejected. Cost equals missed interest income.

The baseline assumptions are:

- 40% recovery rate
- 60% loss-given-default
- 4% opportunity cost on rejected good loans

Expected loss at default 0.50 threshold

	model	threshold_type	threshold	recovery_rate	interest_rate	accuracy	precision_BAD_1
0	XGBoost	default_0.50	0.5000	0.4000	0.0400	0.9211	0.8140
1	Decision Tree	default_0.50	0.5000	0.4000	0.0400	0.8345	0.5595
2	Gradient Boosting	default_0.50	0.5000	0.4000	0.0400	0.9161	0.8485
3	AdaBoost	default_0.50	0.5000	0.4000	0.0400	0.9083	0.8362
4	Random Forest	default_0.50	0.5000	0.4000	0.0400	0.8993	0.7827
5	Logistic Regression	default_0.50	0.5000	0.4000	0.0400	0.8814	0.7599

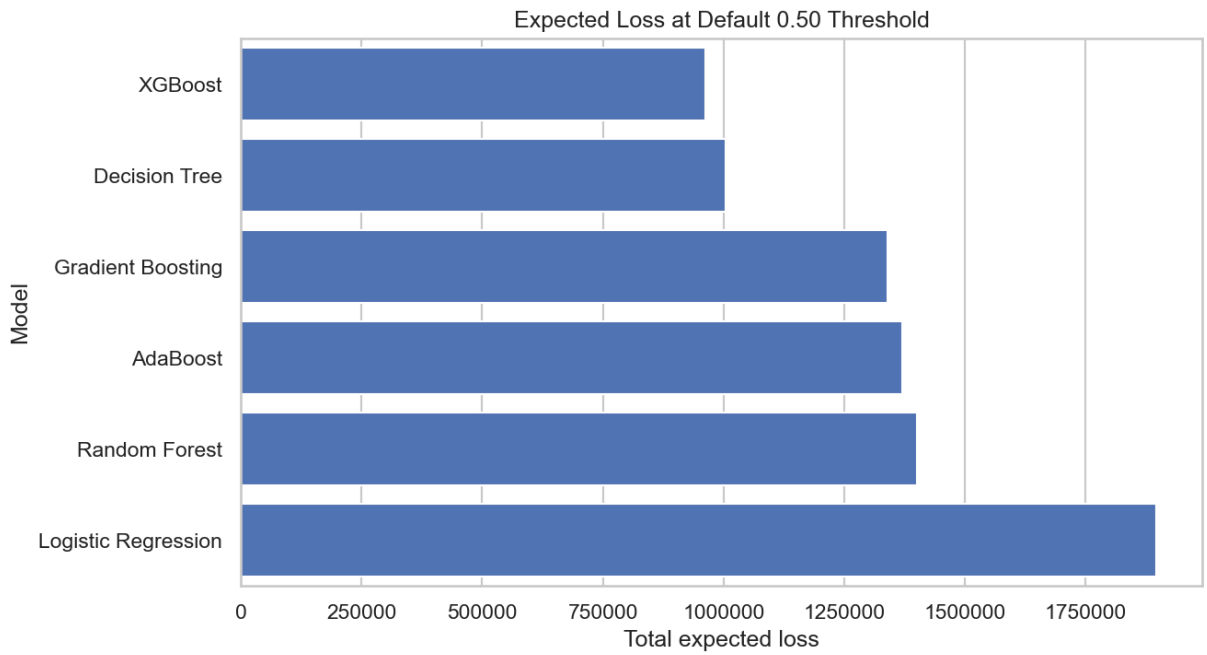
Expected loss after threshold optimisation

	model	threshold_type	threshold	recovery_rate	interest_rate	accuracy	precisio
0	XGBoost	loss_minimising_threshold	0.0600	0.4000	0.0400	0.8221	
1	Random Forest	loss_minimising_threshold	0.0800	0.4000	0.0400	0.7640	
2	Gradient Boosting	loss_minimising_threshold	0.0100	0.4000	0.0400	0.8702	
3	AdaBoost	loss_minimising_threshold	0.4600	0.4000	0.0400	0.7651	
4	Decision Tree	loss_minimising_threshold	0.1100	0.4000	0.0400	0.7220	
5	Logistic Regression	loss_minimising_threshold	0.0400	0.4000	0.0400	0.5626	

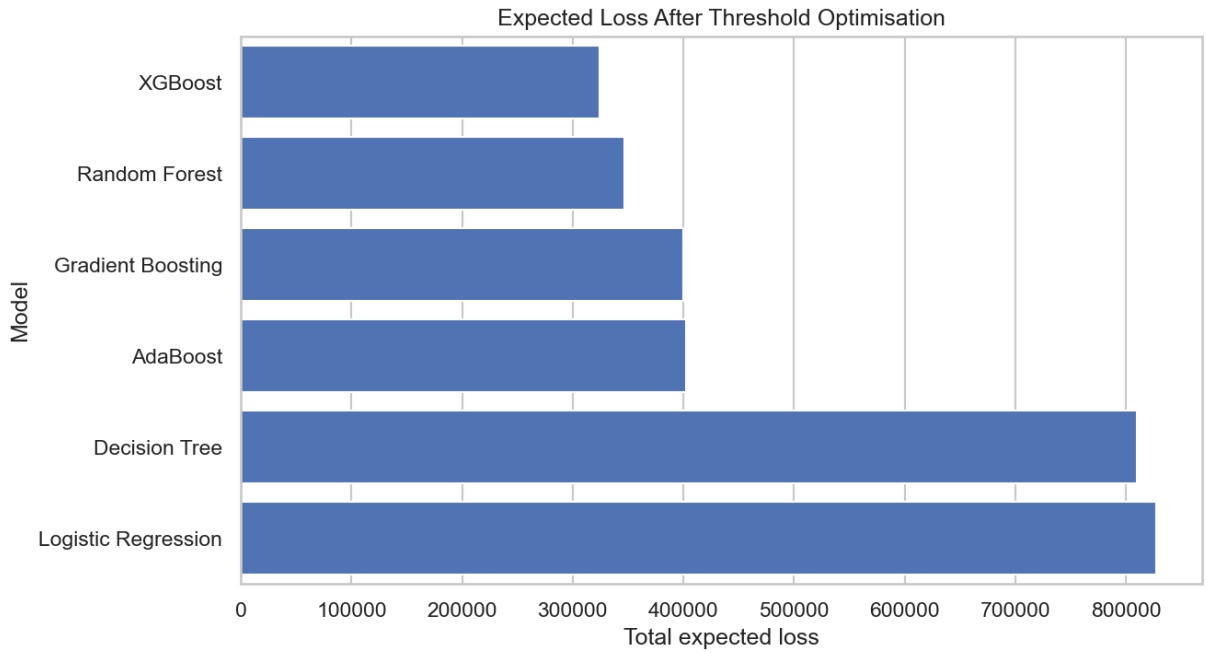
Threshold optimisation summary

	model	default_threshold	best_threshold	default_expected_loss	optimised_expected_loss
0	XGBoost	0.5000	0.0600	963,020.0000	324,052.0000
1	Random Forest	0.5000	0.0800	1,400,768.0000	346,560.0000
2	Gradient Boosting	0.5000	0.0100	1,338,948.0000	399,904.0000
3	AdaBoost	0.5000	0.4600	1,370,952.0000	402,308.0000
4	Decision Tree	0.5000	0.1100	1,004,348.0000	810,016.0000
5	Logistic Regression	0.5000	0.0400	1,896,780.0000	827,732.0000

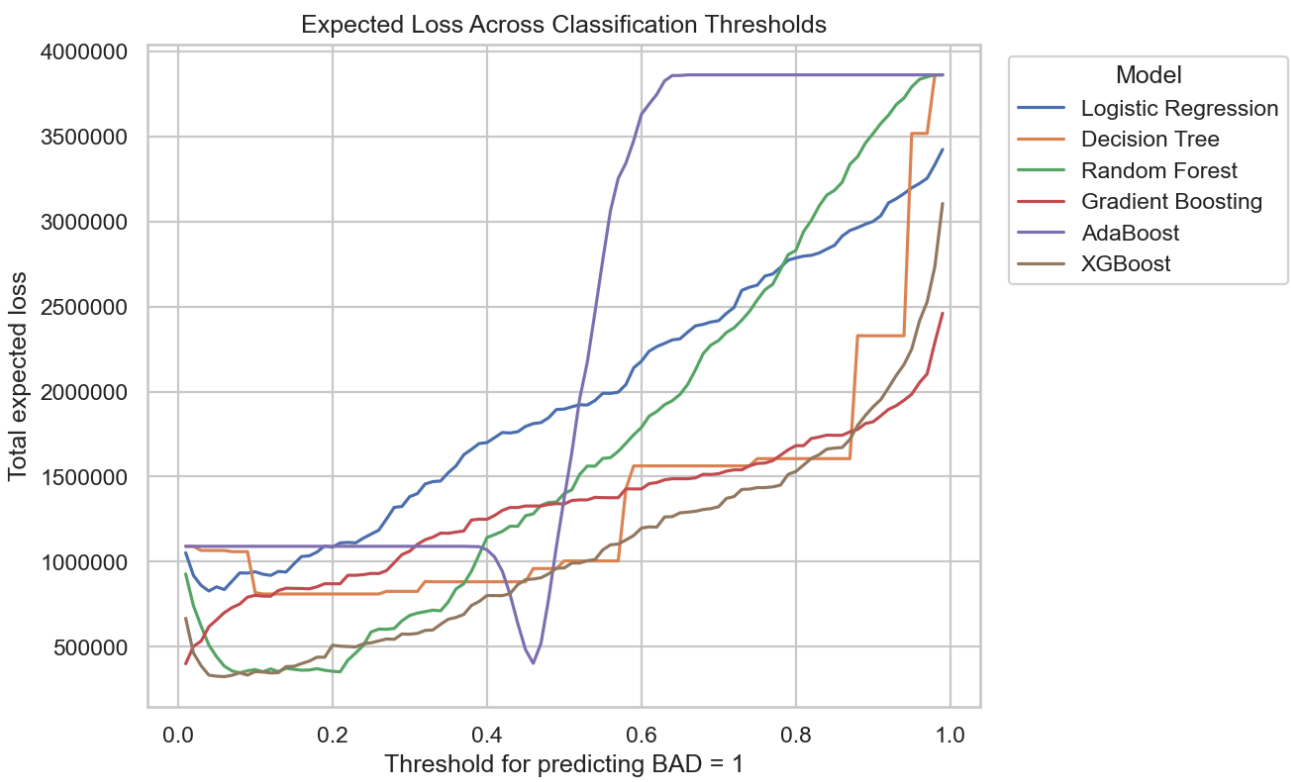
Expected loss at default 0.50 threshold



Expected loss after threshold optimisation



Expected loss across thresholds



Interpretation. XGBoost has the lowest expected loss in the baseline comparison and remains the lowest after threshold optimisation. Threshold optimisation materially lowers expected loss because the default threshold of `0.50` is not aligned with asymmetric lending costs. The economic analysis therefore adds a practical decision layer beyond standard predictive metrics.

Robustness Checks

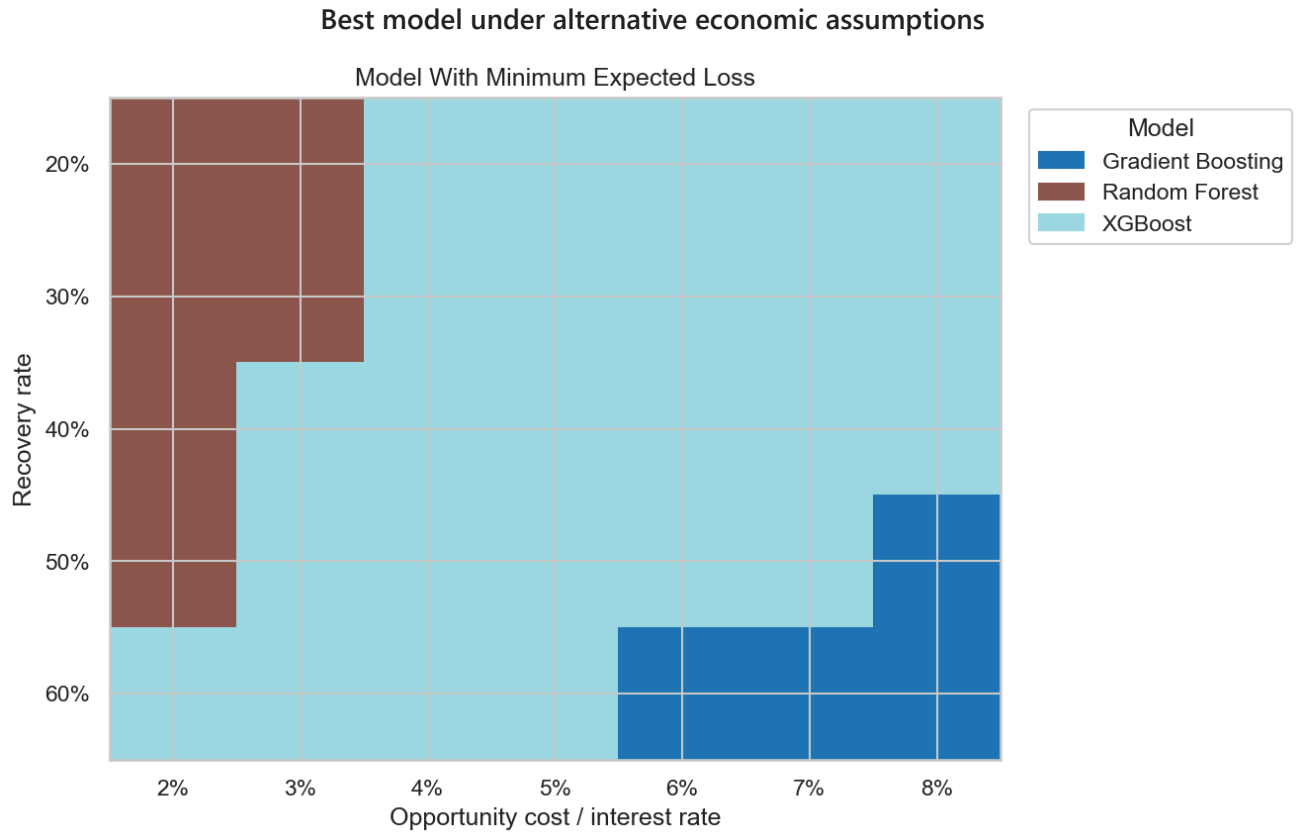
The robustness analysis varies recovery-rate and interest-rate assumptions. This tests whether the preferred model depends heavily on one narrow economic scenario.

Best-model frequency across robustness grid

	model	winning_grid_cells
0	XGBoost	25
1	Random Forest	6
2	Gradient Boosting	4

Best model by recovery-rate / interest-rate assumption

	model	threshold	recovery_rate	interest_rate	false_negative_loss	false_positive_loss	total_
0	Random Forest	0.0800	0.2000	0.0200	72,000.0000	146,280.0000	
1	Random Forest	0.0800	0.2000	0.0300	72,000.0000	219,420.0000	
2	XGBoost	0.0600	0.2000	0.0400	150,960.0000	210,832.0000	
3	XGBoost	0.0600	0.2000	0.0500	150,960.0000	263,540.0000	
4	XGBoost	0.0600	0.2000	0.0600	150,960.0000	316,248.0000	
5	XGBoost	0.0600	0.2000	0.0700	150,960.0000	368,956.0000	
6	XGBoost	0.0600	0.2000	0.0800	150,960.0000	421,664.0000	
7	Random Forest	0.0800	0.3000	0.0200	63,000.0000	146,280.0000	
8	Random Forest	0.0800	0.3000	0.0300	63,000.0000	219,420.0000	
9	XGBoost	0.0600	0.3000	0.0400	132,090.0000	210,832.0000	
10	XGBoost	0.0600	0.3000	0.0500	132,090.0000	263,540.0000	
11	XGBoost	0.0600	0.3000	0.0600	132,090.0000	316,248.0000	
12	XGBoost	0.0600	0.3000	0.0700	132,090.0000	368,956.0000	
13	XGBoost	0.0600	0.3000	0.0800	132,090.0000	421,664.0000	
14	Random Forest	0.0800	0.4000	0.0200	54,000.0000	146,280.0000	



Interpretation. The preferred model is fairly robust, but not universal. XGBoost wins most assumption combinations, while Random Forest and Gradient Boosting become preferred under some recovery-rate and opportunity-cost settings. This is a useful result: model selection should be reviewed under business assumptions rather than treated as purely technical.

SHAP Model Interpretation

The SHAP analysis explains the best-performing main predictive model, which is XGBoost by ROC-AUC. This analysis is separate from the expected-loss threshold optimisation: it explains the predictive model itself.

For this XGBoost setup, SHAP values are reported in raw-margin / log-odds space. Positive SHAP values push predictions toward higher default risk; negative values push predictions toward lower default risk.

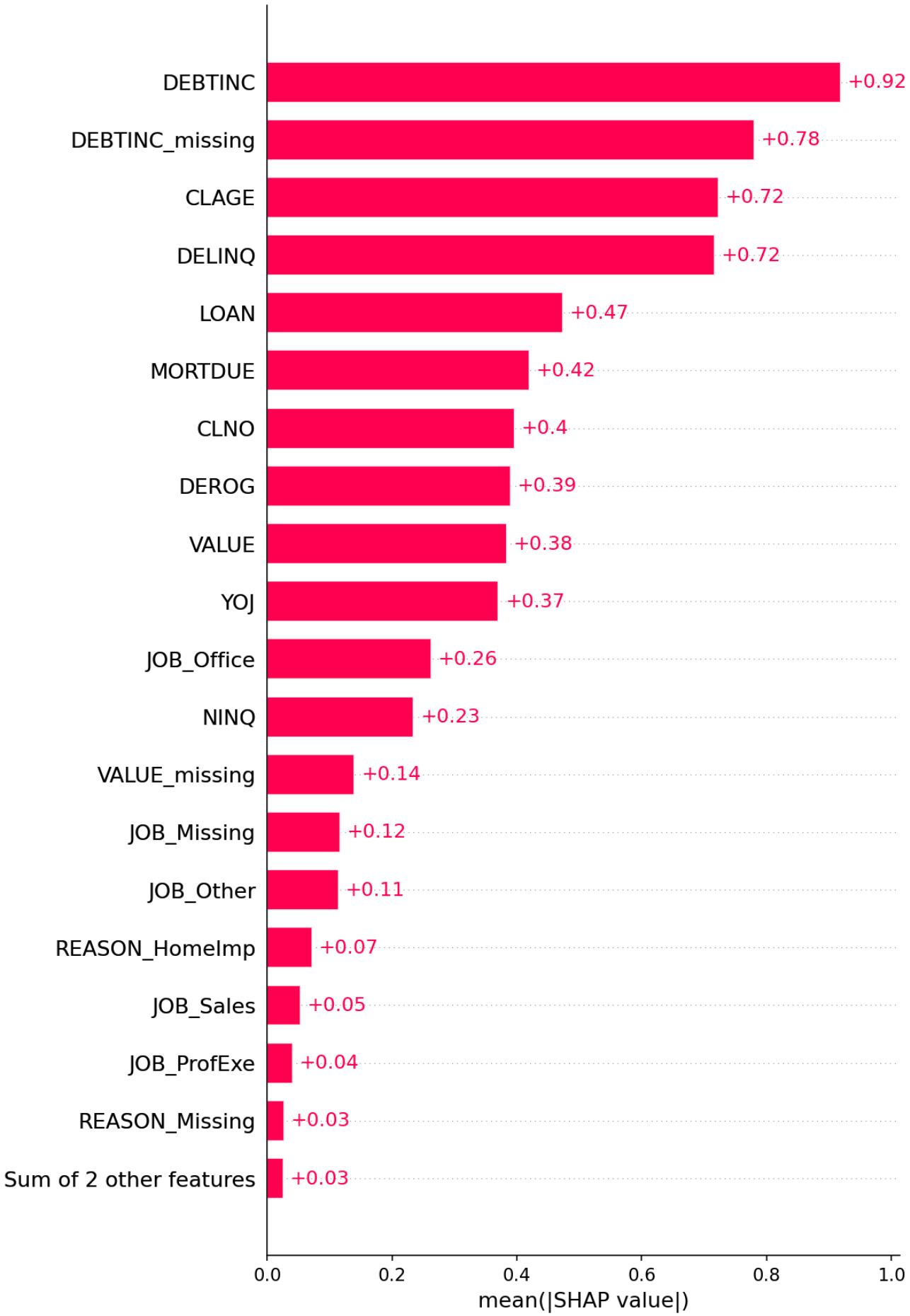
Model selection metrics for SHAP

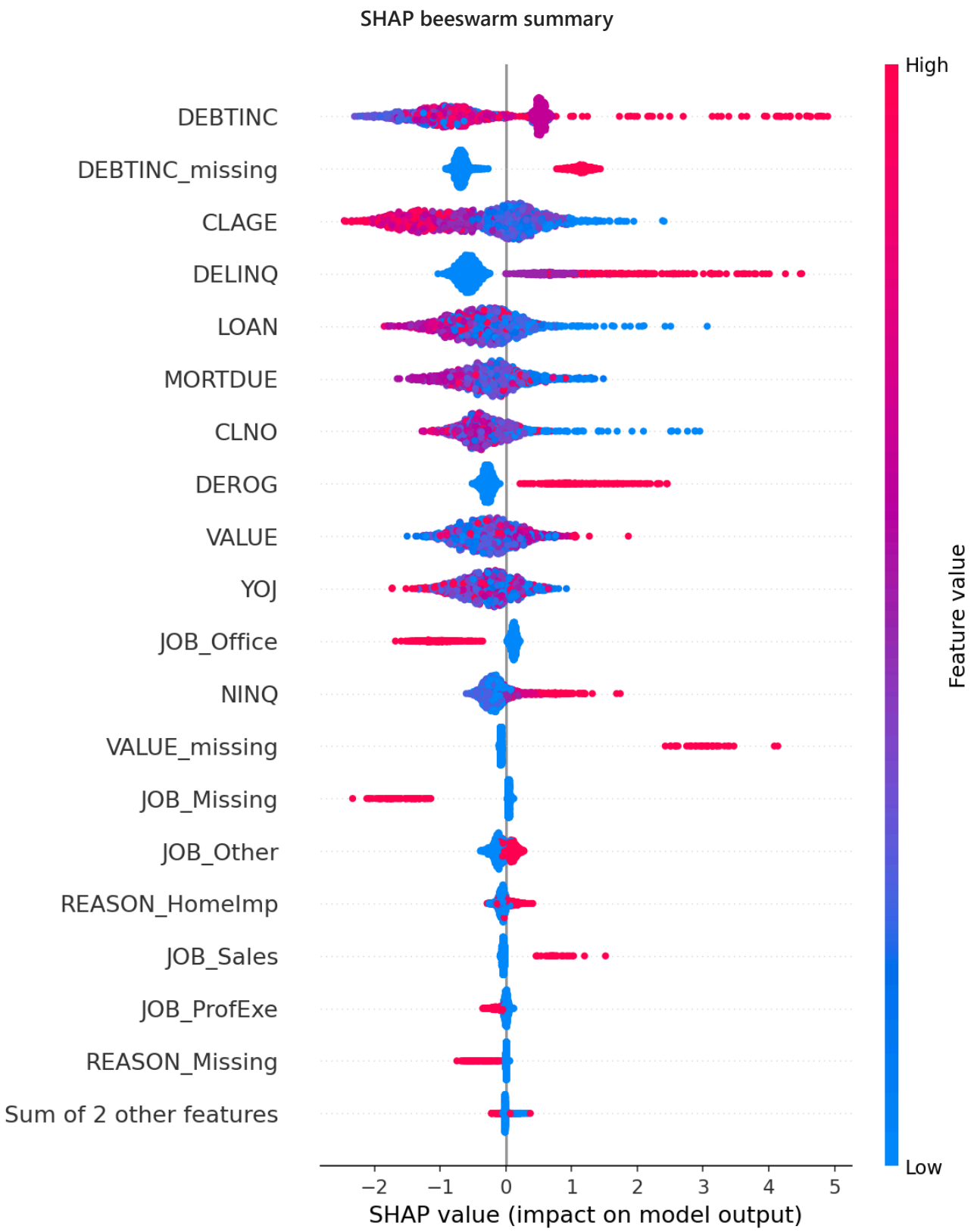
	model_label	accuracy	precision_BAD_1	recall_BAD_1	f1_BAD_1	roc_auc	average_precision
0	Logistic Regression	0.8798	0.7219	0.6471	0.6824	0.8903	NaN
1	Decision Tree	0.8697	0.6845	0.6443	0.6638	0.8835	NaN
2	Random Forest	0.9049	0.7416	0.8039	0.7715	0.9554	0.8348
3	Gradient Boosting	0.9178	0.7720	0.8347	0.8022	0.9633	0.8975
4	AdaBoost	0.9083	0.7749	0.7619	0.7684	0.9506	0.8641
5	XGBoost	0.9239	0.7962	0.8319	0.8137	0.9641	0.8971

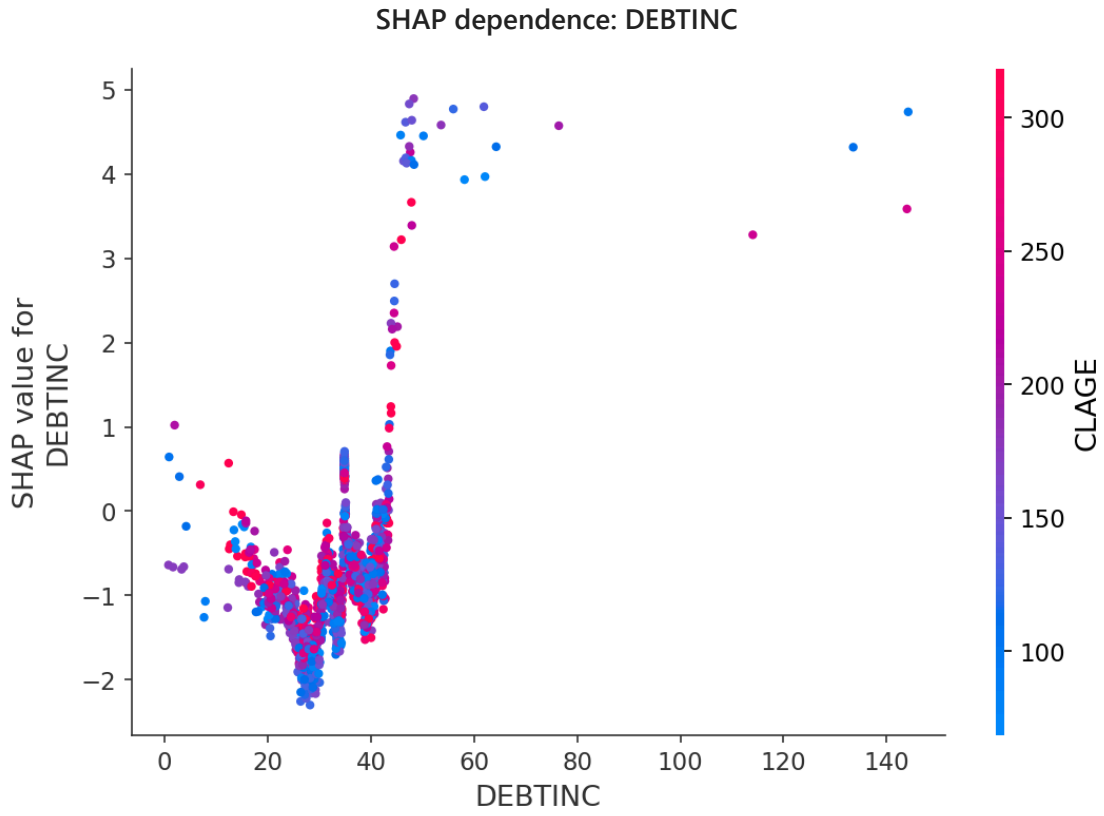
Top SHAP features

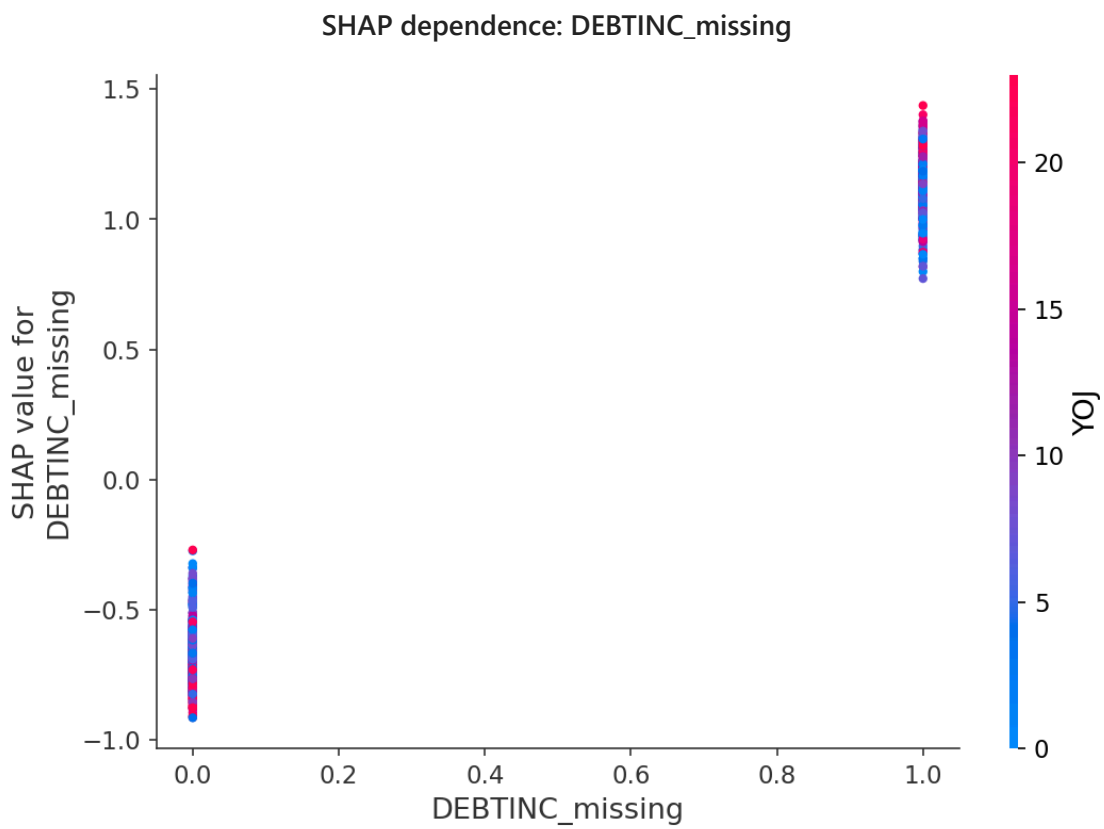
	rank	feature	clean_feature_label	mean_abs_shap	mean_shap
0	1	DEBTINC	DEBTINC	0.9187	-0.5146
1	2	DEBTINC_missing	DEBTINC missing	0.7798	-0.2883
2	3	CLAGE	CLAGE	0.7225	-0.4154
3	4	DELINQ	DELINQ	0.7167	-0.2153
4	5	LOAN	LOAN	0.4735	-0.3299
5	6	MORTDUE	MORTDUE	0.4195	-0.2511
6	7	CLNO	CLNO	0.3964	-0.2825
7	8	DEROG	DEROG	0.3897	-0.1113
8	9	VALUE	VALUE	0.3833	-0.2431
9	10	YOJ	YOJ	0.3703	-0.2567
10	11	JOB_Office	JOB Office	0.2628	-0.0665
11	12	NINQ	NINQ	0.2347	-0.1074
12	13	VALUE_missing	VALUE missing	0.1399	-0.0113
13	14	JOB_Missing	JOB Missing	0.1166	-0.0264
14	15	JOB_Other	JOB Other	0.1141	-0.0429

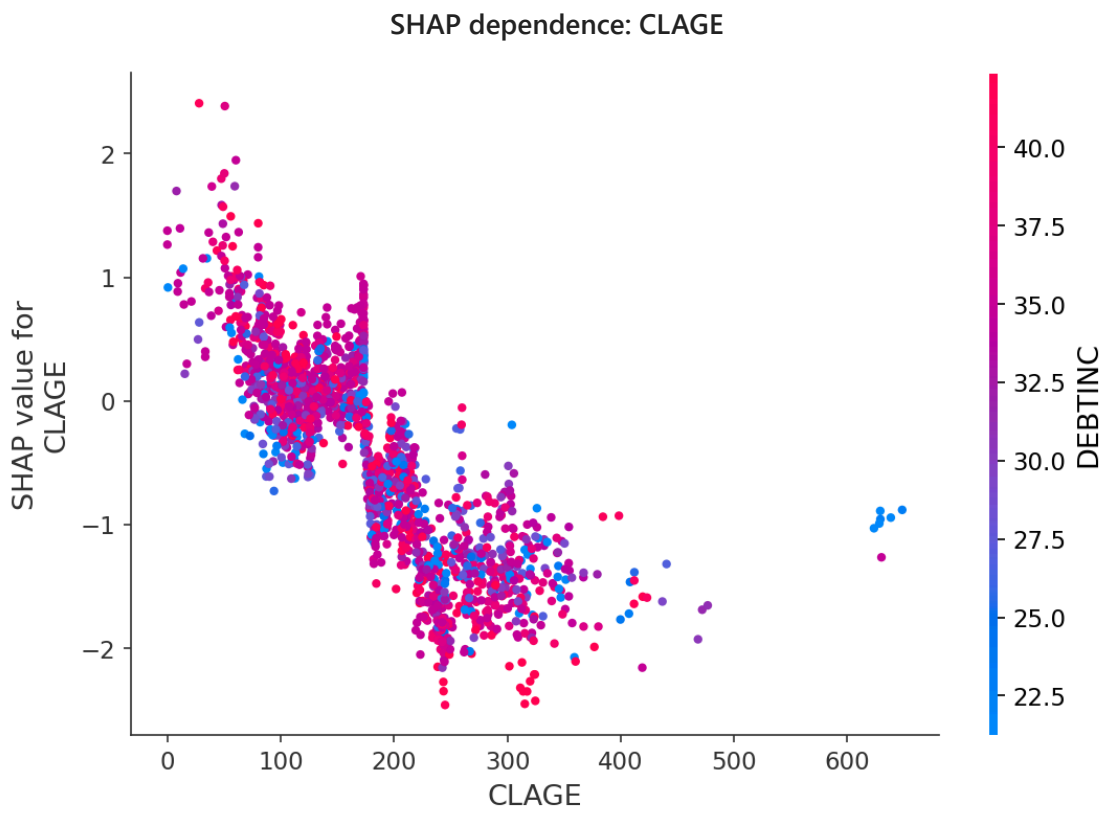
SHAP mean absolute importance

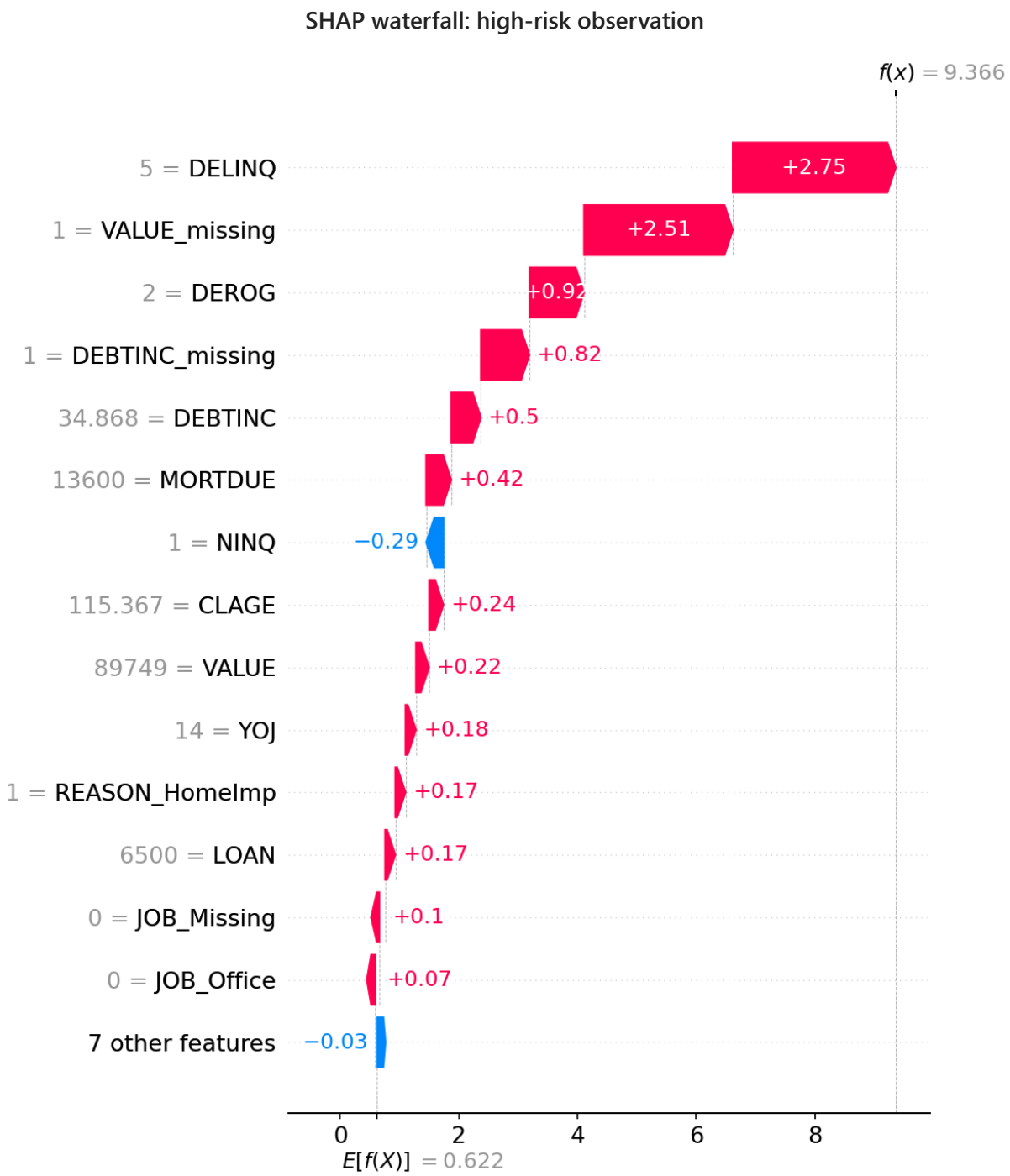

DEBTINC
+0.92
DEBTINC_missing
+0.78
CLAGE
+0.72
DELINQ
+0.72
LOAN
+0.47
MORTDUE
+0.42
CLNO
+0.4
DEROG
+0.39
VALUE
+0.38
YOJ
+0.37
JOB_Office
+0.26
NINQ
+0.23
VALUE_missing
+0.14
JOB_Missing
+0.12
JOB_Other
+0.11
REASON_HomeImp
+0.07
JOB_Sales
+0.05
JOB_ProfExe
+0.04
REASON_Missing
+0.03
Sum of 2 other features
+0.03
0.0
0.2
0.4
0.6
0.8
1.0
mean(|SHAP value|)


SHAP beeswarm summary
DEBTINC
DEBTINC_missing
CLAGE
DELINQ
LOAN
MORTDUE
CLNO
DEROG
VALUE
YOJ
JOB_Office
NINQ
VALUE_missing
JOB_Missing
JOB_Other
REASON_HomeImp
JOB_Sales
JOB_ProfExe
REASON_Missing
Sum of 2 other features
−2
−1
0
1
2
3
4
5
SHAP value (impact on model output)
High
Feature value
Low


SHAP dependence: DEBTINC
SHAP value for DEBTINC
DEBTINC
CLAGE
5
4
3
2
1
0
−1
−2
0
20
40
60
80
100
120
140
300
250
200
150
100


SHAP dependence: DEBTINC_missing
SHAP value for DEBTINC_missing
DEBTINC_missing
YOJ
1.5
1.0
0.5
0.0
−0.5
−1.0
0.0
0.2
0.4
0.6
0.8
1.0
20
15
10
5
0


SHAP dependence: CLAGE
SHAP value for CLAGE
CLAGE
DEBTINC
2
1
0
−1
−2
0
100
200
300
400
500
600
40.0
37.5
35.0
32.5
30.0
27.5
25.0
22.5



Interpretation. The strongest drivers are debt-to-income burden, missing DTI information, credit age, delinquencies, loan amount, mortgage due, number of credit lines, derogatory reports, property value, and years on job. These are predictive associations rather than causal claims.

Main Conclusions

1. **Best main predictive model:** XGBoost has the highest ROC-AUC and strongest overall balance of predictive metrics among the saved model comparisons.

2. **Best expected-loss model:** XGBoost also has the lowest expected loss under the baseline economic assumptions after threshold optimisation.
3. **Key trade-off:** lowering the classification threshold catches more likely defaulters but increases false positives. This can still reduce total expected loss when false negatives are much more expensive.
4. **Robustness:** XGBoost remains preferred across most recovery-rate and interest-rate scenarios, but Random Forest and Gradient Boosting win under some assumptions.
5. **Interpretability:** SHAP shows that the model relies mainly on debt burden, missing debt information, credit history, delinquencies, collateral-related variables, and loan exposure.

Recommendations

A lender should use the XGBoost model as a decision-support benchmark, not as an automatic approval system. The operating threshold should be selected using explicit business assumptions about default losses, recovery rates, and opportunity costs.

Recommended practical approach:

- Use XGBoost as the primary predictive model.
- Set the decision threshold based on expected-loss analysis rather than defaulting to `0.50`.
- Monitor false negatives closely because missed defaulters are costly.
- Use SHAP explanations for model governance and human review.
- Reassess the threshold when recovery rates, loan pricing, or portfolio risk appetite change.

Limitations and Possible Extensions

Important limitations:

- The dataset is historical and may not represent current borrower behaviour.
- Model outputs are predictive associations, not causal explanations.
- Fairness and compliance analysis has not yet been fully developed.
- Probability calibration should be assessed before production use.
- Threshold optimisation here uses the held-out test set for business analysis; production deployment should use validation or cross-validation policy tuning.

Possible extensions:

- Probability calibration using Platt scaling or isotonic regression.
- Fairness analysis across protected or proxy-sensitive variables.
- Cost-sensitive training rather than post-training threshold optimisation.
- Stability testing over time.
- A lightweight dashboard for comparing model decisions and economic assumptions.

Appendix: Script Outputs Used

This notebook uses outputs generated by the following scripts:

- `project_descr.py` : descriptive analysis and EDA outputs
- `project_logit.py` : logistic regression outputs
- `project_tree.py` : decision tree outputs
- `project_rf.py` : random forest outputs
- `project_gb.py` : Gradient Boosting outputs
- `project_ada.py` : AdaBoost outputs
- `project_xg.py` : XGBoost outputs
- `project_loss.py` : expected-loss and robustness outputs
- `project_shap.py` : SHAP interpretation outputs

All paths in this notebook are relative to the project root.